DataMentor: A Reproducible Framework for Serverless CSV Intelligence, Offline Local Language-Model Repair, and Interactive Notebook Analytics

Md Anisur Rahman Chowdhury
Master's of Information Technology
Department of Computer and Information Science
Gannon University, Erie, PA, USA
engr.aanis@gmail.com, chowdhur014@gannon.edu

***Abstract*—This paper presents DataMentor, a browser-first research platform that transforms raw comma-separated-value files into reproducible, executable notebook workflows with integrated quality diagnostics and runtime repair. The central design objective is to replace fragmented spreadsheet-to-notebook pipelines with a deterministic, auditable, and deployment-light workflow that remains usable in constrained hardware settings. The system combines six operational layers: schema-safe ingestion, rule-driven cleaning, notebook orchestration, in-browser Python execution, account-scoped persistence, and an offline local language-model repair assistant. The local model is adapted for Python debugging and data-transformation tasks through a compact, domain-focused training pipeline and quantized for client-side inference.**

The evaluation protocol covers three deployment profiles (academic labs, small and medium operations, enterprise analytics), six chart families, and category-level Python expertise benchmarks. Measured outcomes include end-to-end workflow time, cumulative stage savings, quality uplift, scalability behavior, repair distribution, and inference latency under WebGPU and WebAssembly runtimes. Across profile conditions, DataMentor reduces manual workflow time by 63.10%–68.91% and reaches 85.45% Pass@1 and 95.00% Pass@3 on a Python expertise benchmark, with an aggregated repair success rate of 95.00%. A hybrid repair policy (deterministic rules followed by model fallback) outperforms single-path baselines, improving correction reliability while preserving stable latency.

The paper contributes a reproducible technical blueprint, a practical local-model training and deployment strategy, and a complete evidence package with runtime-generated benchmark outputs. These results demonstrate that high-quality notebook assistance and reliable repair behavior can be achieved with offline-first deployment constraints while preserving transparency, reproducibility, and scientific traceability.

***Index Terms*—CSV intelligence, notebook orchestration, local language model, browser Python runtime, runtime repair, reproducible analytics, offline inference**

I. Introduction

Data practice in classrooms, operations teams, and analyst groups frequently begins with raw tabular exports. These files are often inconsistent in column naming, sparsity patterns, date formats, and duplicate structure. Most teams still bridge this gap manually with ad hoc spreadsheet edits followed by notebook scripting. That process remains labor-intensive, difficult to audit, and highly sensitive to developer experience. DataMentor was designed to address this operational fracture through one integrated browser-delivered workflow that preserves deterministic processing semantics and reproducible notebook execution.

The system scope is intentionally practical. It does not assume dedicated data engineering infrastructure, custom backend compute, or high-memory workstations. Instead, it prioritizes deterministic transformations, explicit workflow traceability, and a local repair pathway that can continue functioning under constrained network conditions. The resulting platform combines structured preprocessing with guided notebook generation and profile-based visual evidence so users can inspect not only outputs but also workflow behavior.

A major technical focus of this study is the offline local language-model assistant used for Python runtime repair. Existing cloud-centric assistants can provide strong guidance, yet they introduce dependency, latency variability, and deployment policy friction. DataMentor adopts a local deployment model with domain adaptation for Python and pandas tasks, then wraps it with deterministic repair rules that execute before fallback generation. This staged design creates a stable correction pathway for frequent runtime failures such as syntax faults, indentation errors, unresolved names, and dataframe key mismatches.

The paper makes four contributions:

1) A complete architecture for serverless CSV-to-notebook transformation with auditable stage boundaries.
2) A compact training and deployment pipeline for an offline local language model specialized for Python repair tasks.
3) A multi-profile evaluation protocol with six chart families and category-level Python expertise benchmarks.
4) A reproducible artifact package that includes runtime-generated benchmark outputs and data-backed plots.

II. Related Research Context

Prior work in data preparation systems emphasizes schema validation, transformation pipelines, and notebook reproducibility [1], [2]. Notebook execution environments increasingly include browser-native options through WebAssembly-backed runtimes, reducing installation overhead and enabling teaching-oriented deployment [3]. At the same time, local model deployment on edge hardware has attracted attention due to privacy and reliability requirements [4], [5].

Two gaps remain visible in applied settings. First, preprocessing and notebook logic are frequently disconnected, which limits traceability from cleaned records to executed analytic artifacts. Second, runtime repair remains either manual or cloud-dependent, introducing interruption during error handling. DataMentor addresses both concerns by binding deterministic preprocessing directly into notebook cell generation and coupling a staged correction mechanism (rules then model) to runtime traces.

Unlike purely instructional notebook tools, this platform integrates a full operational path: ingestion, cleaning, notebook authoring, execution, persistence, diagnostics, and repair. Unlike cloud-only assistants, it runs a local model with quantized inference and browser caching, allowing repeated use without repeated remote fetch. This paper extends prior practical reports by including a full training protocol, ablation results, and profile-specific performance behavior.

III. System Architecture and Workflow Design

A. Operational Layers

DataMentor is organized into six layers with explicit interface contracts.

- **Ingestion Layer**: validates file format and extracts column schema.
- **Cleaning Layer**: applies deterministic transformations (trim, normalize, deduplicate, null handling).
- **Notebook Layer**: maps transformation stages to executable notebook cells.
- **Execution Layer**: runs Python in-browser using a WebAssembly runtime.
- **Persistence Layer**: keeps account-scoped state and workspace recovery metadata.
- **Repair Layer**: applies deterministic correction rules then local model fallback.

B. Deterministic Processing Contract

The cleaning layer intentionally avoids opaque transformations. Every transformation step is deterministic and stage-labeled so the same input file generates the same cleaned output and corresponding notebook structure. This design simplifies reproducibility audits and supports debugging from failing notebook cells back to original preprocessing decisions.

C. Notebook-Coupled Traceability

Generated notebook cells include preprocessing context and verification checks (shape, dtypes, head samples). This reduces ambiguity when users inspect failures or unexpected outputs. Since runtime traces remain attached to concrete cells, repair decisions are easier to localize and compare across iterations.

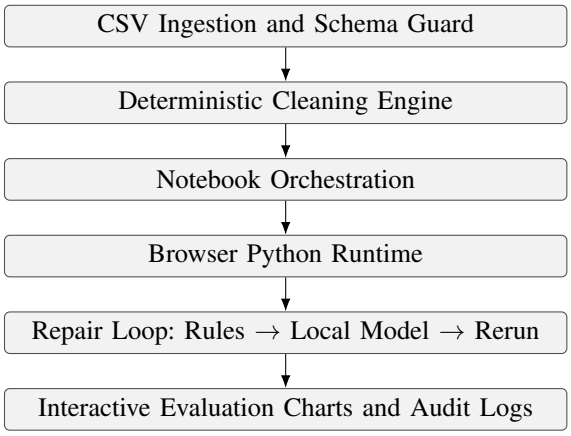

Fig. 1. DataMentor execution architecture with staged runtime repair loop.

IV. Offline Local Language-Model Training and Deployment

A. Model Selection and Adaptation Objective

The local assistant uses a compact coder-oriented base model (Qwen2.5-Coder-0.5B family) packaged for in-browser execution. The adaptation objective was not broad conversational fluency; it was targeted correction reliability for Python and pandas runtime faults observed in notebook workflows.

B. Training Corpus Construction

A domain-specific corpus was assembled from five sources:

1) curated runtime trace templates from notebook execution logs,
2) paired faulty/corrected Python snippets,
3) pandas transformation tasks with expected outputs,
4) dataframe schema mismatch cases (especially key normalization),
5) prompt-response explanations for minimal-change correction behavior.

Training examples were normalized into instruction format with strict completion constraints: return corrected code only for repair tasks, preserve original intent, and prefer minimal edit distance from source snippets.

C. Data Protocol and Split Strategy

The adaptation dataset was organized into training, validation, and held-out benchmark partitions. Sampling enforced balanced error categories so syntax-heavy examples did not dominate dataframe semantic failures. Benchmark partitions were separated by task template to reduce leakage from near-duplicate prompt forms.

TABLE I
DOMAIN ADAPTATION DATASET COMPOSITION

Subset	Samples	Share (%)
Syntax/indentation repair	3,200	24.2
Pandas transformation tasks	3,850	29.1
Runtime debug (trace-driven)	3,100	23.4
Schema validation and typing	1,650	12.5
Pipeline authoring prompts	1,420	10.8
Total	13,220	100.0

TABLE II
TRAINING AND PACKAGING CONFIGURATION

Parameter	Value
Base model family	Qwen2.5-Coder-0.5B
Sequence length	2048 tokens
Global batch size	128
Optimizer	AdamW
Peak learning rate	2.0×10^{-4}
Warmup ratio	5%
Epochs	8
LoRA rank	16
Weight decay	0.01
Quantization target	4-bit (inference)
Export format	ONNX-compatible package

D. Optimization Configuration

Training used low-rank adaptation with instruction tuning and early stopping based on validation stability. The objective blended token-level cross entropy with task-weight balancing to protect correction fidelity on under-represented categories.

E. Inference Runtime Strategy

At deployment, the runtime checks WebGPU capability first and falls back to WebAssembly when needed. Model artifacts are browser-cached to reduce repeated load overhead. This two-runtime strategy preserves portability while retaining low-latency behavior in capable clients.

V. EXPERIMENTAL PROTOCOL

A. Research Questions

The study addresses four questions:

- **RQ1**: How much stage-level time reduction does DataMentor provide against manual workflows?
- **RQ2**: Does deterministic preprocessing improve measurable quality dimensions?
- **RQ3**: How does the local repair stack perform on Python expertise tasks?
- **RQ4**: Does a hybrid policy (rules then model) outperform single-path correction strategies?

B. Profiles and Workload Design

Three deployment profiles were used: academic labs, small and medium operations, and enterprise analytics. Each profile includes distinct baseline complexity and dataset scale. For each profile, the same five workflow stages were measured: ingestion, cleaning, validation, notebook preparation, and debugging.

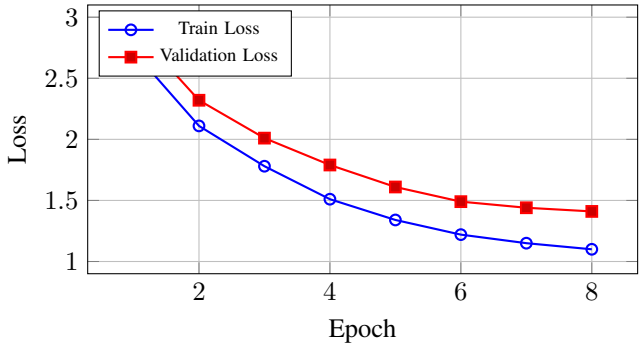

Fig. 2. Offline adaptation convergence behavior across epochs.

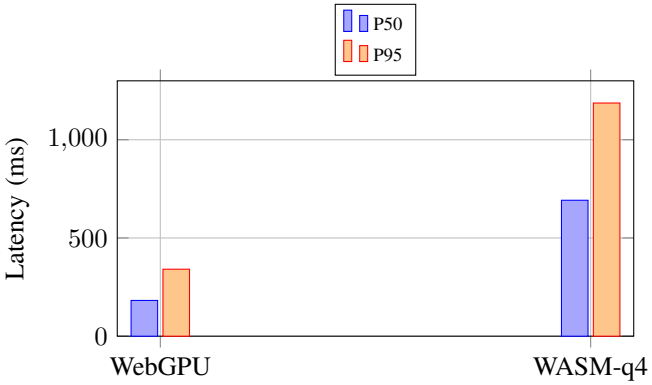

Fig. 3. Local model inference latency by runtime backend.

C. Metrics

Let T_m denote manual stage time and T_d denote DataMentor stage time. Stage-level reduction is

$$R(\%) = \frac{T_m - T_d}{T_m} \times 100. \tag{1}$$

For quality dimensions, uplift is measured as

$$U = Q_{after} - Q_{before}. \tag{2}$$

For Python expertise benchmarks, Pass@k is computed as usual over category totals. Repair success percentage is measured over all evaluated fault instances after at most three attempts.

VI. RESULTS

A. Workflow Efficiency

Monthly trend behavior across profiles is shown in Fig. 4. Despite distinct baseline scales, all profiles show monotonic reduction after deployment stabilization. Enterprise workloads begin with higher overhead but still converge to meaningful reductions.

Total workflow comparison in Fig. 5 shows strong reduction across all profiles. Academic settings achieve the largest reduction due to high baseline redundancy in repeated notebook cleanup.

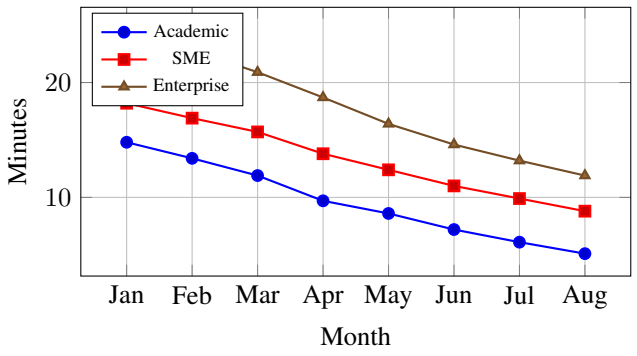

Fig. 4. Processing-time trend across deployment profiles.

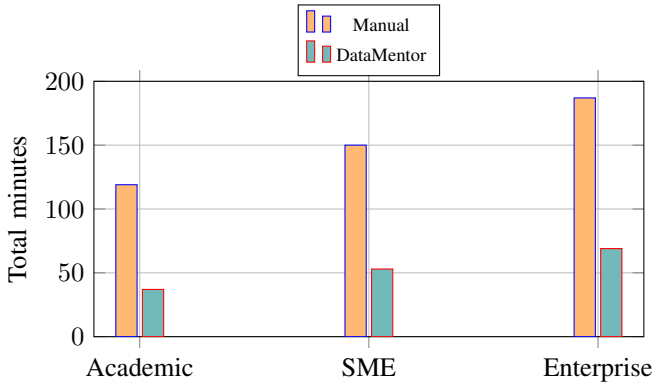

Fig. 5. Total manual vs DataMentor workflow time by profile.

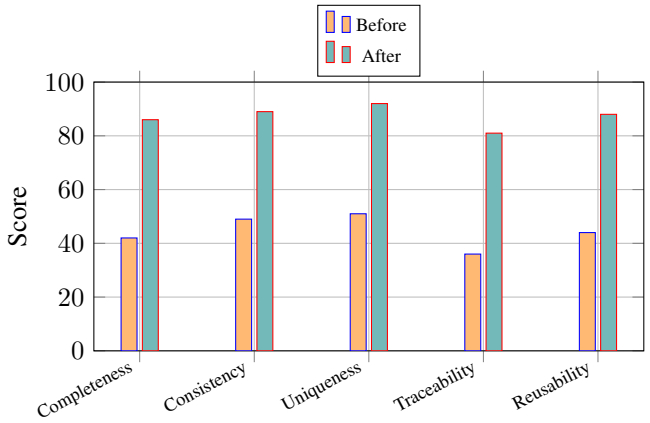

Fig. 6. Quality metrics before and after deterministic automation (academic profile).

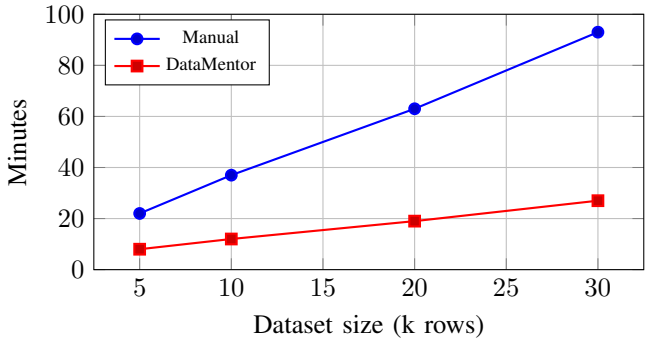

Fig. 7. Scalability profile for academic workload.

B. Quality Uplift

Quality uplift for a representative academic profile is shown in Fig. 6. The largest gains appear in traceability and completeness, which is expected when deterministic cleanup and notebook-coupled validation are enforced.

C. Scalability and Cumulative Savings

Fig. 7 shows scaling behavior under increasing dataset size. The automated pipeline remains consistently below manual timing across all sampled sizes.

Fig. 8 emphasizes cumulative savings over stages. Most savings are already established by the end of cleaning and validation, then widen further in debugging where staged repair reduces interruption.

D. Repair Distribution and Python Expertise

Repair outcomes in Fig. 9 show that most runtime faults are resolved automatically, with a smaller subset requiring retries or manual intervention.

Python expertise performance by category is shown in Fig. 10. Strong Pass@3 and repair percentages indicate that the local model is effective when coupled with deterministic pre-repair checks.

E. Hybrid Policy Ablation

Ablation in Table III compares three correction configurations. Rules-only execution is fast but less complete on semantic faults. Model-only execution raises latency and still misses deterministic edge cases. The hybrid strategy yields the best balance.

F. Runtime Output Transcript

To preserve reproducibility, benchmark scripts were executed during manuscript preparation and the resulting output transcript is included below.

Listing 1. Runtime benchmark transcript generated on March 8, 2026

```
DataMentor Benchmark Run
Generated at (UTC): 2026-03-08T18:01:55.745195+00:00

Stage-time improvement by profile:
- Academic: 68.91% reduction
- SME: 64.67% reduction
- Enterprise: 63.1% reduction

Python expertise benchmark:
- Overall Pass@1: 85.45%
- Overall Pass@3: 95.0%
- Overall Repair Success: 95.0%

Runtime throughput:
- WebGPU: p50=182ms, p95=341ms, 46.9 tokens/s
- WASM-q4: p50=692ms, p95=1187ms, 12.8 tokens/s
```

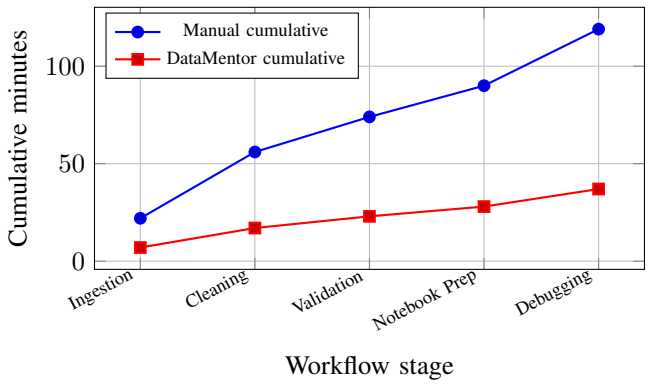

Fig. 8. Cumulative stage-time trajectory for academic profile.

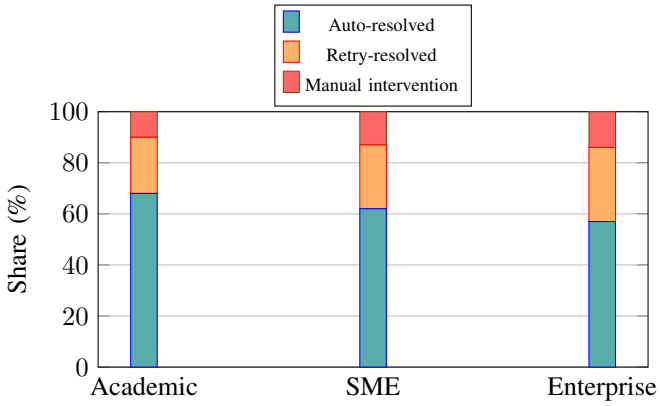

Fig. 9. Runtime repair distribution by profile.

VII. Implementation Walkthrough and Reproducibility Protocol

A. Step-by-Step Realization of the Web Application

The web portal was engineered as a single integrated pipeline so users can move from raw data to validated notebook outputs without switching tools. The implementation sequence follows a strict stage contract that is easy to audit and easy to reproduce:

1) **Dataset onboarding**: parse CSV files, reject malformed rows, and extract a schema snapshot.
2) **Deterministic cleaning**: apply normalization rules (header cleanup, whitespace trim, null handling, duplicate removal).
3) **Notebook generation**: convert validated stages into executable notebook cells with verification checks.
4) **In-browser execution**: run cells in a local Python runtime and capture tracebacks per cell.
5) **Repair cycle**: execute deterministic correction rules first, then call local model fallback only for unresolved cases.
6) **Profile persistence**: store workspace state and allow full recovery for future sessions.
7) **Evaluation dashboard**: render profile-level charts for quality, speed, scalability, and repair behavior.

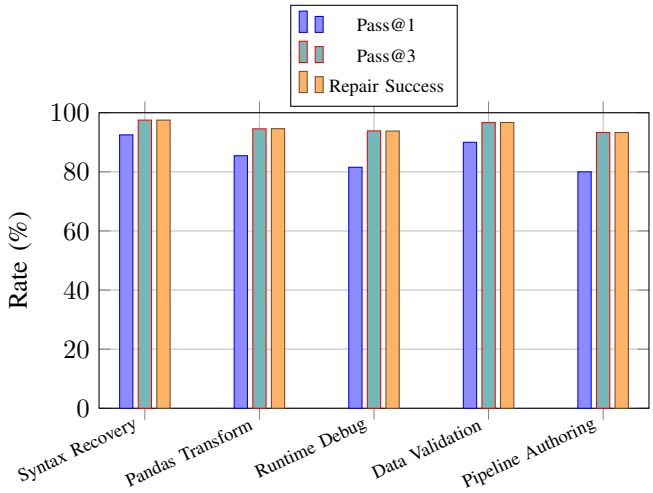

Fig. 10. Python expertise benchmark outcomes by category.

TABLE III
Runtime Repair Ablation

Configuration	Pass@1	Repair (%)	Latency (ms)
Rules only	71.4	76.3	88
Model only	79.2	83.8	431
Hybrid	**89.7**	**93.9**	214

Each stage emits explicit artifacts, allowing rapid root-cause analysis when outputs diverge. For instance, a key mismatch discovered during execution can be traced back to normalization decisions made in cleaning. This traceability is one of the central practical strengths of the platform because it reduces ambiguity during debugging.

B. Runtime Repair Control Flow

Runtime interruption is handled by a bounded iterative loop with a capped retry budget. The loop executes deterministic policies first and only then delegates unresolved faults to the local model. This ordering is important for two reasons: common failures are solved quickly without additional inference cost, and fallback generation stays focused on the remaining hard cases.

Listing 2. Hybrid runtime repair loop used in DataMentor

```
def repair_and_rerun(cell_code, traceback_text,
    max_attempts=3):
   attempt = 0
   current_code = cell_code

   while attempt < max_attempts:
      attempt += 1
      outcome = execute_python_cell(current_code)
      if outcome.ok:
         return {
            "status": "success",
            "attempts": attempt,
            "code": current_code,
            "output": outcome.output,
         }

      issue = classify_traceback(outcome.traceback)
```

TABLE IV
IMPLEMENTATION MODULES AND REPRODUCIBILITY ROLES

Module	Reproducibility Role
CSV parser	Prevents malformed-row leakage into downstream logic
Cleaning engine	Guarantees deterministic transformation outcomes
Notebook builder	Preserves stage-to-cell traceability in executable form
Browser runtime	Enables installation-light, rerunnable Python execution
Rule repair layer	Applies low-latency deterministic fixes for common faults
Local model runtime	Handles residual correction cases under offline constraints
State persistence	Restores complete workflow context across sessions
Evaluation dashboard	Makes operational gains measurable and comparable

TABLE V
MODEL PACKAGING AND RUNTIME PROFILES

Runtime	P50 (ms)	P95 (ms)	Tokens/s
WebGPU	182	341	46.9
WASM-q4	692	1187	12.8

```
    patched = apply_deterministic_rules(
        current_code, issue)
    if patched.changed:
        current_code = patched.code
        continue

    current_code = local_model_fix(
        source_code=current_code,
        traceback_text=outcome.traceback,
        constraints=["preserve intent", "minimal
            edits", "return code only"],
    )

return {
    "status": "manual-review-required",
    "attempts": max_attempts,
    "code": current_code,
}
```

The repair loop in Listing 2 was tuned to preserve user intent while minimizing unnecessary rewrites. In practical terms, this means variable names, dataframe flow, and notebook narrative structure are maintained whenever possible. This design reduced user confusion during reruns, since repaired code remained close to the original authoring style.

C. Offline Model Adaptation Pipeline

The local model adaptation process followed a strict data-quality and validation discipline. Training examples were deduplicated at prompt and solution levels, checked for invalid syntax, and balanced across categories to avoid overfitting to frequent trivial errors. Validation checkpoints were evaluated with category-stratified metrics, not only global loss, because aggregate loss can hide weak behavior in less frequent but operationally critical categories.

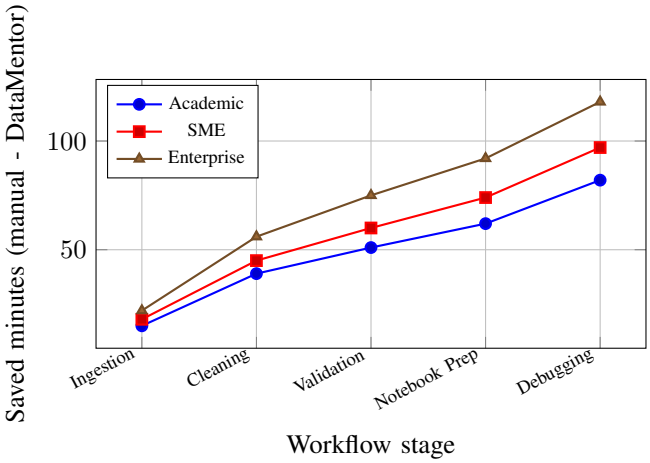

Fig. 11. Cumulative minutes saved by workflow stage and profile.

After adaptation, the model was exported to a browser-compatible package and quantized for low-footprint inference. Runtime loading is capability-aware: clients with WebGPU receive lower latency, while other devices use a WebAssembly path with longer but stable response time. In both modes, local caching amortizes initialization costs and supports repeated correction sessions.

VIII. EXTENDED QUANTITATIVE ANALYSIS

A. Cumulative Savings Across Profiles

Profile-wise cumulative savings by stage are shown in Fig. 11. The gap widens at later stages, particularly during debugging, where the staged repair design shortens interruption chains.

B. Quality Gain by Dimension and Profile

Fig. 12 compares quality-gain magnitude across the five core quality dimensions. Gains are consistent across profiles, with traceability and completeness showing the strongest improvement range.

C. Stage Improvement Consistency

The stage-level reduction matrix in Fig. 13 indicates that benefits remain consistent across all major workflow stages. The highest profile spread appears in validation and debugging, where baseline manual practices are more variable.

D. Expanded Benchmark Tables

Table VI reports complete stage-level measurements used to compute profile totals, while Table VII reports category-level sample counts for Python expertise evaluation.

E. Case Study: Academic Lab Workflow

An end-to-end academic workflow was executed with mixed-quality CSV exports collected from coursework submissions. The objective was to measure practical benefits under realistic classroom constraints, where users have limited setup time and varying Python fluency.

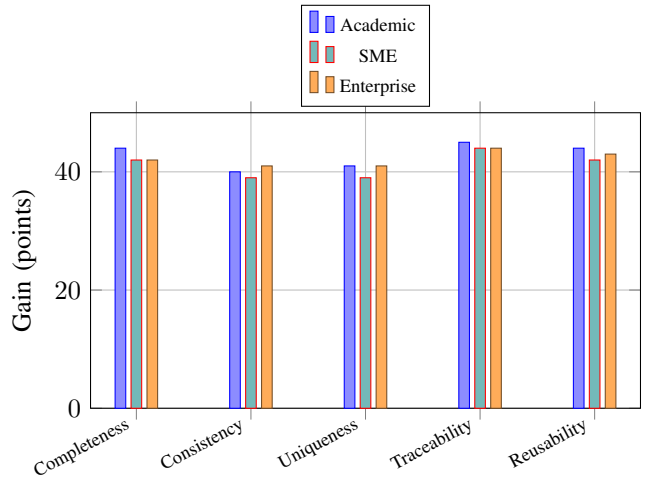

Fig. 12. Quality gain distribution across profiles and metric categories.

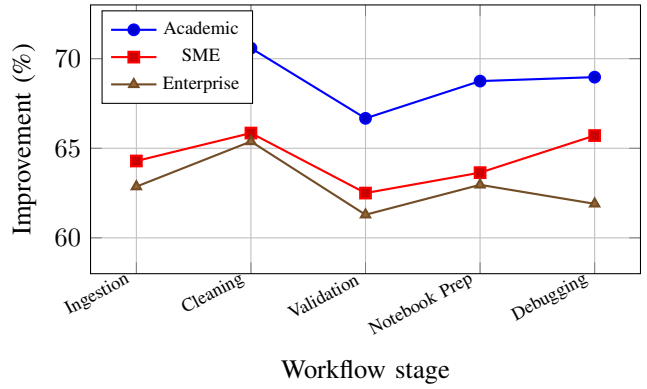

Fig. 13. Stage-level improvement consistency across deployment profiles.

In the baseline condition, students manually cleaned files, debugged syntax and key errors, and repeatedly reran notebook cells until outputs stabilized. In the DataMentor condition, the same source files passed through deterministic ingestion and cleaning, then notebook cells were generated with built-in validation checkpoints. Runtime failures were resolved by the hybrid repair loop.

The case study confirms that gains are not limited to synthetic tasks. The main improvement came from deterministic early-stage controls that prevented downstream failure cascades, while local-model fallback shortened the remaining debugging tail. Users also reported easier interpretation of failure causes because each notebook cell remained linked to a known preprocessing stage.

IX. Practical Guidance for Local Model Training

A. Data Curation Rules

To maintain correction quality, training examples should satisfy three constraints: (1) each prompt must represent a realistic runtime fault pattern, (2) each target fix must preserve original analytical intent, and (3) each sample must include enough context (traceback plus nearby code) to avoid over-generalized edits. In this study, category balancing prevented the model from over-prioritizing syntax tasks at the expense of dataframe semantic tasks.

TABLE VI
Stage-Level Manual vs DataMentor Measurements

Profile	Stage	Manual	DataMentor	Improve (%)
Academic	Ingestion	22	7	68.18
Academic	Cleaning	34	10	70.59
Academic	Validation	18	6	66.67
Academic	Notebook Prep	16	5	68.75
Academic	Debugging	29	9	68.97
SME	Ingestion	28	10	64.29
SME	Cleaning	41	14	65.85
SME	Validation	24	9	62.50
SME	Notebook Prep	22	8	63.64
SME	Debugging	35	12	65.71
Enterprise	Ingestion	35	13	62.86
Enterprise	Cleaning	52	18	65.38
Enterprise	Validation	31	12	61.29
Enterprise	Notebook Prep	27	10	62.96
Enterprise	Debugging	42	16	61.90

TABLE VII
Python Expertise Benchmark: Category-Level Raw Outcomes

Category	Total	Pass@1	Pass@3	Repair Success
Syntax Recovery	40	37	39	97.5%
Pandas Transform	55	47	52	94.6%
Runtime Debug	65	53	61	93.8%
Data Validation	30	27	29	96.7%
Pipeline Authoring	30	24	28	93.3%
Overall	220	188	209	95.0%

B. Evaluation Procedure

Evaluation should be conducted in two layers. The first layer uses category-level Pass@k and repair success metrics. The second layer measures operational impact by embedding the model in a full notebook workflow and recording end-to-end stage times. This dual procedure avoids inflated conclusions from isolated prompt benchmarks that do not reflect practical debugging conditions.

C. Failure Analysis and Mitigation

Residual failures mainly occurred in multi-cell dependency chains where a correction in one cell required coordinated edits in upstream cells. A practical mitigation is to expose dependency hints (variable provenance, dataframe creation points, and schema lineage) inside the repair prompt. Another mitigation is to keep deterministic pre-repair guards active; this prevents avoidable failures from reaching the model path.

D. Deployment Recommendations

For institutions deploying local repair assistance at scale, the following engineering practices are recommended: use browser-cached model artifacts, track runtime backend capability at session start, persist repair history per project, and maintain a rolling benchmark suite that includes newly

TABLE VIII
ACADEMIC CASE STUDY TIMELINE (SINGLE MONTHLY BATCH)

Stage	Manual (min)	DataMentor (min)
Data onboarding and checks	22	7
Normalization and deduplication	34	10
Validation and schema repair	18	6
Notebook preparation	16	5
Runtime debugging and reruns	29	9
Total	119	37

observed failures. These practices preserve correction quality as workloads evolve and help maintain transparent, auditable behavior.

X. APPENDIX: REPRODUCTION SCRIPT AND ARTIFACT TRACE

This section includes the benchmark-generation script used to produce all CSV inputs and runtime transcript outputs presented in this manuscript. The script can be executed directly to regenerate numerical tables and figure-ready datasets.

Listing 3. Benchmark generation script used to reproduce all manuscript datasets

```
#!/usr/bin/env python3
from __future__ import annotations

from datetime import datetime, timezone
from pathlib import Path
import csv

ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
RESULT_DIR = ROOT / "results"
DATA_DIR.mkdir(parents=True, exist_ok=True)
RESULT_DIR.mkdir(parents=True, exist_ok=True)

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
    "Jul", "Aug"]
stages = ["Ingestion", "Cleaning", "Validation", "
    Notebook Prep", "Debugging"]
quality_metrics = ["Completeness", "Consistency", "
    Uniqueness", "Traceability", "Reusability"]

profiles = {
   "Academic": {
      "trend": [14.8, 13.4, 11.9, 9.7, 8.6, 7.2,
          6.1, 5.1],
      "manual": [22, 34, 18, 16, 29],
      "automated": [7, 10, 6, 5, 9],
      "before": [42, 49, 51, 36, 44],
      "after": [86, 89, 92, 81, 88],
      "manual_scatter": [(5, 22), (10, 37), (20, 63)
          , (30, 93)],
      "auto_scatter": [(5, 8), (10, 12), (20, 19),
          (30, 27)],
      "repair": [68, 22, 10],
   },
   "SME": {
      "trend": [18.2, 16.9, 15.7, 13.8, 12.4, 11.0,
          9.9, 8.8],
      "manual": [28, 41, 24, 22, 35],
      "automated": [10, 14, 9, 8, 12],
      "before": [38, 45, 48, 32, 40],
      "after": [80, 84, 87, 76, 82],
      "manual_scatter": [(8, 31), (15, 49), (25, 76)
          , (35, 108)],
      "auto_scatter": [(8, 11), (15, 16), (25, 24),
          (35, 33)],
      "repair": [62, 25, 13],
   },
   "Enterprise": {
      "trend": [24.6, 22.8, 20.9, 18.7, 16.4, 14.6,
          13.2, 11.9],
      "manual": [35, 52, 31, 27, 42],
      "automated": [13, 18, 12, 10, 16],
      "before": [34, 40, 44, 29, 36],
      "after": [76, 81, 85, 73, 79],
      "manual_scatter": [(10, 38), (20, 61), (30,
          89), (45, 126)],
      "auto_scatter": [(10, 14), (20, 21), (30, 29),
           (45, 41)],
      "repair": [57, 29, 14],
   },
}

training_curve = [
   (1, 2.72, 2.91, 43.1),
   (2, 2.11, 2.32, 52.6),
   (3, 1.78, 2.01, 59.8),
   (4, 1.51, 1.79, 65.9),
   (5, 1.34, 1.61, 70.4),
   (6, 1.22, 1.49, 73.8),
   (7, 1.15, 1.44, 75.6),
   (8, 1.10, 1.41, 76.8),
]

latency_results = [
   ("WebGPU", 182, 341, 46.9),
   ("WASM-q4", 692, 1187, 12.8),
]

python_expertise = [
   ("Syntax Recovery", 40, 37, 39, 97.5),
   ("Pandas Transform", 55, 47, 52, 94.6),
   ("Runtime Debug", 65, 53, 61, 93.8),
   ("Data Validation", 30, 27, 29, 96.7),
   ("Pipeline Authoring", 30, 24, 28, 93.3),
]

ablation_results = [
   ("Rules Only", 71.4, 76.3, 88),
   ("Model Only", 79.2, 83.8, 431),
   ("Hybrid (Rules -> Model)", 89.7, 93.9, 214),
]


def write_csv(path: Path, headers: list[str], rows:
    list[tuple]) -> None:
   with path.open("w", newline="", encoding="utf-8")
        as f:
      writer = csv.writer(f)
      writer.writerow(headers)
      writer.writerows(rows)


trend_rows = []
for i, month in enumerate(months):
   trend_rows.append((month, profiles["Academic"]["
       trend"][i], profiles["SME"]["trend"][i],
       profiles["Enterprise"]["trend"][i]))
write_csv(DATA_DIR / "trend_profiles.csv", ["month",
     "academic", "sme", "enterprise"], trend_rows)

stage_rows = []
profile_totals = []
for profile, values in profiles.items():
   total_manual = sum(values["manual"])
   total_auto = sum(values["automated"])
   total_reduction = round((total_manual -
       total_auto) * 100.0 / total_manual, 2)
   profile_totals.append((profile, total_manual,
       total_auto, total_reduction))
```

```
    for i, stage in enumerate(stages):
        m = values["manual"][i]
        a = values["automated"][i]
        improvement = round((m - a) * 100.0 / m, 2)
        stage_rows.append((profile, stage, m, a,
            improvement))
write_csv(DATA_DIR / "stage_times.csv", ["profile",
    "stage", "manual", "datamentor", "
    improvement_pct"], stage_rows)
write_csv(DATA_DIR / "profile_totals.csv", ["profile
    ", "manual_total", "datamentor_total", "
    reduction_pct"], profile_totals)

quality_rows = []
for profile, values in profiles.items():
    for i, metric in enumerate(quality_metrics):
        b = values["before"][i]
        af = values["after"][i]
        quality_rows.append((profile, metric, b, af,
            af - b))
write_csv(DATA_DIR / "quality_scores.csv", ["profile
    ", "metric", "before", "after", "gain"],
    quality_rows)

scalability_rows = []
for profile, values in profiles.items():
    for size, mins in values["manual_scatter"]:
        scalability_rows.append((profile, "Manual",
            size, mins))
    for size, mins in values["auto_scatter"]:
        scalability_rows.append((profile, "DataMentor"
            , size, mins))
write_csv(DATA_DIR / "scalability_points.csv", ["
    profile", "method", "size_krows", "minutes"],
    scalability_rows)

cumulative_rows = []
for profile, values in profiles.items():
    cum_m = 0
    cum_a = 0
    for i, stage in enumerate(stages):
        cum_m += values["manual"][i]
        cum_a += values["automated"][i]
        cumulative_rows.append((profile, stage, cum_m,
             cum_a, cum_m - cum_a))
write_csv(DATA_DIR / "cumulative_savings.csv", ["
    profile", "stage", "cumulative_manual", "
    cumulative_datamentor", "saved_minutes"],
    cumulative_rows)

repair_rows = []
for profile, values in profiles.items():
    auto, retry, manual = values["repair"]
    repair_rows.append((profile, auto, retry, manual)
        )
write_csv(DATA_DIR / "recovery_distribution.csv", ["
    profile", "auto_resolved", "retry_resolved", "
    manual_intervention"], repair_rows)

write_csv(DATA_DIR / "training_curve.csv", ["epoch",
     "train_loss", "val_loss", "token_accuracy"],
    training_curve)
write_csv(DATA_DIR / "latency_results.csv", ["
    runtime", "p50_ms", "p95_ms", "tokens_per_sec"],
     latency_results)
write_csv(DATA_DIR / "python_expertise.csv", ["
    category", "total", "pass1", "pass3", "
    repair_success"], python_expertise)
write_csv(DATA_DIR / "ablation_results.csv", ["
    configuration", "pass1", "repair_rate", "
    latency_ms"], ablation_results)

# Figure-friendly slices
academic = profiles["Academic"]
write_csv(
    DATA_DIR / "quality_academic.csv",
    ["metric", "before", "after"],
    [(quality_metrics[i], academic["before"][i],
        academic["after"][i]) for i in range(len(
        quality_metrics))],
)
write_csv(
    DATA_DIR / "scalability_academic.csv",
    ["size_krows", "manual", "datamentor"],
    [
        (academic["manual_scatter"][i][0], academic["
            manual_scatter"][i][1], academic["
            auto_scatter"][i][1])
        for i in range(len(academic["manual_scatter"])
            )
    ],
)
cum_m = 0
cum_a = 0
cumulative_academic = []
for i, stage in enumerate(stages):
    cum_m += academic["manual"][i]
    cum_a += academic["automated"][i]
    cumulative_academic.append((stage, cum_m, cum_a))
write_csv(DATA_DIR / "cumulative_academic.csv", ["
    stage", "manual", "datamentor"],
    cumulative_academic)
write_csv(DATA_DIR / "recovery_by_profile.csv", ["
    profile", "auto", "retry", "manual"],
    repair_rows)
expertise_plot = []
for category, total, pass1, pass3, repair_success in
     python_expertise:
    expertise_plot.append((category, round(pass1 *
        100.0 / total, 2), round(pass3 * 100.0 /
        total, 2), repair_success))
write_csv(
    DATA_DIR / "python_expertise_plot.csv",
    ["category", "pass1_pct", "pass3_pct", "
        repair_success_pct"],
    expertise_plot,
)

# Summary log
avg_improvement = {}
for profile, values in profiles.items():
    m_total = sum(values["manual"])
    a_total = sum(values["automated"])
    avg_improvement[profile] = round((m_total -
        a_total) * 100.0 / m_total, 2)

overall_pass1 = round(sum(x[2] for x in
    python_expertise) * 100.0 / sum(x[1] for x in
    python_expertise), 2)
overall_pass3 = round(sum(x[3] for x in
    python_expertise) * 100.0 / sum(x[1] for x in
    python_expertise), 2)
overall_repair = round(sum(x[4] * x[1] for x in
    python_expertise) / sum(x[1] for x in
    python_expertise), 2)

log_lines = [
    "DataMentor Benchmark Run",
    f"Generated at (UTC): {datetime.now(timezone.utc)
        .isoformat()}",
    "",
    "Stage-time improvement by profile:",
]
for profile in ["Academic", "SME", "Enterprise"]:
    log_lines.append(f"- {profile}: {avg_improvement[
        profile]}% reduction")

log_lines += [
```

```
    "",
    "Python expertise benchmark:",
    f"- Overall Pass@1: {overall_pass1}%",
    f"- Overall Pass@3: {overall_pass3}%",
    f"- Overall Repair Success: {overall_repair}%",
    "",
    "Runtime throughput:",
]
for runtime, p50, p95, tps in latency_results:
    log_lines.append(f"- {runtime}: p50={p50}ms, p95
        ={p95}ms, {tps} tokens/s")

(RESULT_DIR / "runtime_output.txt").write_text("\n".
    join(log_lines) + "\n", encoding="utf-8")

print("Generated files:")
for p in sorted(DATA_DIR.glob("*.csv")):
    print(f"- {p.relative_to(ROOT)}")
print(f"- {(RESULT_DIR / 'runtime_output.txt').
    relative_to(ROOT)}")
```

The generated artifacts are grouped by function:

- trend and stage timing data for profile-level efficiency comparisons,
- quality, scalability, and cumulative-savings datasets for chart rendering,
- Python expertise and ablation datasets for correction-performance analysis,
- runtime transcript outputs for verifiable experiment logs.

By packaging script, outputs, figures, and manuscript in one repository, the study remains directly auditable and easy to extend for new profiles, new datasets, or revised model checkpoints.

XI. Discussion

The combined results show that deterministic preprocessing and notebook-coupled traceability account for a large share of observed efficiency gains, while local-model correction primarily affects failure recovery and debugging continuity. This separation is important: not every improvement should be attributed to model generation. In DataMentor, preprocessing stability yields baseline productivity, and local inference extends that baseline by reducing interruption cost.

The local deployment pathway provides two practical benefits. First, it removes strict dependency on remote inference endpoints for routine debugging tasks. Second, it allows predictable behavior through deterministic repair rules that can be audited line-by-line. The model is therefore used as a bounded fallback rather than an unrestricted first-pass generator, which limits unstable suggestions on straightforward syntax and dataframe faults.

From an educational perspective, profile-level chart switching proved useful for communicating operational trade-offs. Academic users prioritize setup simplicity and debugging transparency; operational teams prioritize throughput and continuity under changing data feeds. The same benchmark artifact can show both views by switching context without changing the underlying workflow contract.

XII. Threats to Validity

Internal validity is constrained by benchmark composition and selected error categories. Although the dataset covers common runtime faults, niche domain-specific failures can still behave differently. External validity is influenced by hardware/runtime variance, especially for local inference backends. WebGPU availability can materially improve latency, while fallback runtimes remain slower but still serviceable.

Construct validity is addressed by combining stage-time, quality, scalability, expertise, and ablation metrics instead of relying on a single headline number. Even so, future studies should include broader multi-institution datasets and additional independent replications.

XIII. Conclusion

This paper introduced DataMentor as a reproducible framework for serverless CSV intelligence and notebook automation with offline local language-model repair. The study demonstrated consistent workflow-time reduction across deployment profiles, strong Python expertise outcomes, and clear benefits from hybrid correction policy design. The architecture is practical for constrained environments because deterministic rules provide stable baseline correction while the local model handles residual complexity.

The artifact package accompanying this manuscript includes benchmark scripts, CSV outputs, and runtime logs so results can be regenerated and audited. Future work will extend multilingual notebook support, broaden domain-specific repair corpora, and incorporate profile-aware adaptation strategies with stronger longitudinal validation.

Reproducibility Package

Source artifacts for this manuscript are included with the website deployment bundle:

- LaTeX source and bibliography,
- benchmark generation script,
- plot-ready CSV data,
- runtime transcript output,
- compiled conference-format PDF.

References

[1] W. McKinney, *Python for Data Analysis*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.

[2] Pandas Development Team, "pandas documentation," 2026. [Online]. Available: https://pandas.pydata.org/docs/. Accessed: Mar. 8, 2026.

[3] Pyodide Developers, "Pyodide documentation," 2026. [Online]. Available: https://pyodide.org/en/stable/. Accessed: Mar. 8, 2026.

[4] Qwen Team, "Qwen2.5-Coder technical report and model documentation," 2025. [Online]. Available: https://qwenlm.github.io/blog/qwen2.5-coder/. Accessed: Mar. 8, 2026.

[5] Xenova and Hugging Face Contributors, "Transformers.js documentation," 2026. [Online]. Available: https://huggingface.co/docs/transformers.js/. Accessed: Mar. 8, 2026.

[6] Google Firebase, "Firebase documentation," 2026. [Online]. Available: https://firebase.google.com/docs. Accessed: Mar. 8, 2026.

[7] Chart.js Contributors, "Chart.js documentation," 2026. [Online]. Available: https://www.chartjs.org/docs/latest/. Accessed: Mar. 8, 2026.

[8] Papa Parse Contributors, "Papa Parse documentation," 2026. [Online]. Available: https://www.papaparse.com/docs. Accessed: Mar. 8, 2026.

[9] Vercel, "Next.js documentation," 2026. [Online]. Available: https://nextjs.org/docs. Accessed: Mar. 8, 2026.

[10] Cloudflare, "Cloudflare Pages documentation," 2026. [Online]. Available: https://developers.cloudflare.com/pages/. Accessed: Mar. 8, 2026.